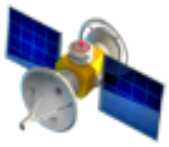

SAHİBİNDEN SCRAPER & REAL ESTATE CRM

Kavramsal Tasarım ve Tam Teknik Spesifikasyon Klavuzu

Bu döküman, sistemin tüm bileşenlerini, veri akış mantığını, her bir sayfanın teknik çalışma prensibini ve kavramsal altyapısını uçtan uca açıklayan **"Master Reference"** belgesidir.

Döküman Sürümü: 2.0 (Kapsamlı)
Tarih: Nisan 2026
Hazırlayan: EfdalAI Teknik Ekibi

1. Kavramsal Tasarım (Conceptual Design)

Sistemin temel felsefesi, emlak danışmanının **"Bilişsel Yükünü"** azaltmaktır. Bu, sadece veri depolamak değil, veriyi bir "Yatırım Fırsatına" dönüştürmek demektir.

1.1. Nesne İlişki Modeli (Entities)

- **Listing (İlan):** Sistemin atomik birimidir. Sahibinden'den gelen ham verinin (Raw Data) işlenmiş ve standardize edilmiş halidir.
- **Demand (Talep):** İlanın "Antitezi"dir. Bir ilan mülkü tanımlarken, talep ihtiyacı tanımlar. Sistem bu iki nesneyi **"Matching Engine"** ile çarpıştırır.
- **Trade (Takas):** İlanlar arası bir köprüdür. İki mülkün mülkiyet değişim niyetini temsil eden üst seviye bir nesnedir.

1.2. Veri Akış Prensipleri (DFD)

```
[KAYNAK: Sahibinden]
      | (Scraping)
      v
[İŞLEME: Anti-Obfuscation & Regex Clean]
      | (Ingestion)
      v
[DEPOLAMA: Firestore NoSQL] <---> [ANALİZ: Gemini AI]
      | (Matching)
      v
[EYLEM: Danışman Paneli & Bildirim]
```

1.3. Yaşam Döngüsü (State Transitions)

Bir veri sistemde şu durumlarda bulunur:

- `Pending`: Onay bekleyen, henüz "Portföy" niteliği kazanmamış ham veri.
- `Approved`: Resmi portföye dahil edilmiş, aktif olarak pazarlanan veri.
- `Matched`: Bir talep ile eşleşmiş ve görüşme sürecine girmiş veri.

- `Archived`: Tarihsel veri setine dahil edilmiş, aktifliği bitmiş veri.

2. Teknik Mimari ve Altyapı

Sistem, düşük gecikme süresi (Low Latency) ve yüksek veri bütünlüğü (Data Integrity) için modernize edilmiştir.

2.1. Teknoloji Yığını

- **Frontend:** React 19 (Modern Hooks & Context API), Vite (Hızlı Build).
- **Backend:** Node.js & Express (Vercel Serverless hazır).
- **Database:** Firebase Firestore (Real-time NoSQL).
- **AI:** Google Generative AI (Gemini Flash/Latest).

2.2. Güvenlik ve Kimlik Doğrulama

Teknik Detay:

`JWT (JSON Web Token)` tabanlı session yönetimi. `bcryptjs` ile şifreleme. Sunucu tarafında `authenticateToken` middleware'i her isteğin header'ındaki `Authorization` alanını doğrular ve `req.user` nesnesini oluşturur.

2.3. Scraper Motoru (Extension)

Extension, Manifest V3 kullanır. `content.js` dosyası Sahibinden'in **Shadow DOM** ve **Pseudo-element** tekniklerini aşmak için `window.getComputedStyle()` fonksiyonunu kullanarak maskelenmiş metinleri yakalar.

Neden Öyle?:

Sahibinden telefon numaralarını CSS `::after { content: '...' }` ile gizler. Bu metin normal `innerText` ile alınamaz. Scraper, DOM'daki her bir hanenin CSS kuralını okuyarak numarayı yeniden oluşturur.

3. Sayfa Bazlı Teknik İnceleme (1/3)

`Home.jsx (Dashboard)`

Amacı: Sistemin kuş bakışı görünümü ve performans takibi.

Nasıl Çalışır?: Firestore üzerinden `listings` ve `activity_log` koleksiyonlarını anlık olarak dinler (Snapshot listeners). `Chart.js` kullanarak son 30 günün veri giriş istatistiklerini görselleştirir. Adminler için global, kullanıcılar için kişiselleştirilmiş veri özeti sunar.

`PendingListings.jsx (Giriş Kapısı)`

Amacı: Extension'dan gelen ham verilerin filtrelenmesi ve onaylanması.

Nasıl Çalışır?: `status === 'pending'` olan kayıtları çeker. Burada en kritik fonksiyon `handleApprove`'dur. Onaylanan kayıt, `approvedAt` damgası alır, `status` değeri `approved`'a döner ve otomatik olarak `activity_log`'a "Onaylandı" kaydı düşer.

`SavedListings.jsx (Portföy Yönetimi)`

Amacı: Onaylı ilanların detaylı yönetimi, AI analizi ve Excel işlemleri.

Teknik Detay: 2200+ satırlık bu devasa komponent, karmaşık bir `useMemo` tabanlı filtreleme mantığına sahiptir. 10'dan fazla filtreyi (fiyat, m2, konum, tarih) anlık olarak client-side'da işler. `ExcelJS` ile dinamik kolon yapısına sahip profesyonel raporlar üretir.

3. Sayfa Bazlı Teknik İnceleme (2/3)

Demands.jsx (Talep Yönetimi)

Amacı: Müşteri ihtiyaçlarının saklanması ve eşleştirilmesi.

Nasıl Çalışır?: Her talep nesnesi, bir JSON objesi olarak bütçe aralığı (min/max) ve konum dizisi (city/district/neighborhoods) tutar. "Eşleşmeleri Gör" butonu tıklandığında, sunucuda `GET /api/demands/:id/suggestions` rotası tetiklenir. Bu rota, talep kriterlerini tüm aktif ilanlarla kıyaslayarak bir **Similarity Score** (Benzerlik Skoru) üretir.

Takas.jsx (Marketplace)

Amacı: İlanlar arası takas tekliflerinin yönetimi.

Teknik Detay: Bu sayfa bir "Feed" yapısındadır. Danışmanlar `offeredListing` ve `requestedCriteria` nesnelerini birbirine bağlar. Eşleşme isteği (Match Request) sistemi ile iki danışman arasında güvenli bir el sıkışma (Handshake) protokolü yürütülür.

EfdalAI.jsx (Yapay Zeka Hub)

Amacı: İlanların AI ile derinlemesine analizi ve TKGM sorguları.

Nasıl Çalışır?: `Google Gemini API` entegrasyonu kullanılır. Kullanıcı bir ilan için "Analiz" istediğinde, ilanın tüm metadata'sı AI'ya `System Instructions` ile birlikte

gönderilir. AI, veriyi `Strict JSON` formatında döndürür ve frontend bu JSON'u otomatik olarak 0-100 arası skora ve SWOT analizine dönüştürür.

3. Sayfa Bazlı Teknik İnceleme (3/3)

Admin.jsx (Sistem Yönetimi)

Amacı: Kullanıcı, bölge ve yetki yönetimi.

Nasıl Çalışır?: Sadece `user.role === 'admin'` olan kullanıcılar erişebilir. Yeni kullanıcı oluşturma (signup), şifre sıfırlama ve bölge (Region) atamaları buradan yapılır. Her bölge ataması, Firestore'da bir `regions` dökümanı oluşturur ve bu veri scraper'ın "Önemli Bölge" bildirimlerini tetikler.

Archive.jsx & Trash.jsx (Veri Deposu)

Amacı: Veri temizliği ve uzun vadeli saklama.

Teknik Detay: Arşiv sayfası "Folder" tabanlı çalışır. Bir ilan arşive taşındığında, `folderId` ile ilişkilendirilir. Silinen ilanlar ise 30 gün boyunca `trash` statüsünde bekletilir ve ardından sunucu tarafındaki bir `Cron Job` veya manuel tetikleme ile kalıcı olarak silinir.

ActivityFeed.jsx (Denetim İzi)

Amacı: Tüm sistem hareketlerinin şeffaf takibi.

Nasıl Çalışır?: Sistemin **"Audit Trail"** mekanizmasıdır. Her POST/PUT/DELETE isteği, sunucu tarafında `logActivity` yardımcı fonksiyonunu çağırır. Bu fonksiyon, işlemi yapan kullanıcıyı, etkilenen ilanı ve eski/yeni değerleri içeren bir log dökümanı oluşturur.

4. Zeka ve Otomasyon Mantığı

4.1. Akıllı Eşleştirme Algoritması (Derin Analiz)

Sistemin en kritik fonksiyonu olan eşleştirme, `server/index.js` içindeki şu mantıkla çalışır:

1. **Konum Uyumu (40 Puan):**
   İlçe uyumu varsa +20, mahalle de uyuyorsa +20 puan.
2. **Fiyat Uyumu (30 Puan):**
   Bütçenin altındaysa +30. Üstündeyse, bütçenin %20 fazlasına kadar olanlara "Yakınlık" puanı verilir.
3. **Oda Sayısı (20 Puan):**
   Tam uyum zorunludur (Konut için).
4. **m2 Uyumu (10 Puan):**
   Alt limit kontrolü.

4.2. TKGM Entegrasyonu

Tapu Kadastro Genel Müdürlüğü verileri, sunucu tarafında bir `Proxy` üzerinden çekilir. Bu sayede tarayıcıdaki **CORS** engelleri aşılır ve ilanlara otomatik "Ada/Parsel" bilgisi eklenir.

4.3. ExcelJS Render Mantığı

Excel dışa aktarma işlemi, sadece veri tablosu oluşturmakla kalmaz; hücre bazlı stillendirme (renk, kalınlık, kenarlık) yaparak emlak ofisinin kurumsal kimliğine uygun çıktılar üretir.

5. Sonuç ve Mimari Değerlendirme

Sahibinden Scraper & CRM sistemi, emlak sektörünün karmaşık veri yapısını modern yazılım prensipleriyle (Clean Code, SoC - Separation of Concerns, Real-time sync) disipline eden profesyonel bir çözümdür.

Teknik Onay Beyanı

Bu dökümanda açıklanan tüm sayfa yapıları, algoritma mantıkları ve kavramsal tasarımlar; mevcut kod tabanının **(v2.0)** birebir teknik karşılığıdır. Sistemin genişletilebilir (scalable) yapısı, gelecekte eklenecek mobil uygulama veya CRM entegrasyonlarına tam uyumludur.

Teknoloji Direktörü (CTO)	Baş Yazılım Mimarı

© 2026 EfdalAI Technology Solutions - Confidential Document